PaperLens: Evidence-Grounded, Reproducibility-Aware Analysis of Research Papers with a Free-Tier Language Model

Author One, Author Two, Author Three, Author Four
Department of Computer Engineering
Sardar Patel Institute of Technology, Mumbai, India
{first.last}@spit.ac.in

***Abstract*—Large language models (LLMs) can summarise a research paper in seconds, but the facts they report come without evidence. For research, an invented number is worse than none. We present *PaperLens*, a system that takes a paper (PDF or link) and returns a structured breakdown in which every extracted item carries the verbatim sentence it came from. Python locates that sentence in the PDF and highlights it, and a second check judges whether it supports the item. On top of this grounded extraction, PaperLens scores reproducibility against a 21-item checklist, verifies numeric claims against the paper's own results tables, discovers the paper's code repository and compares its settings with the paper by static analysis, and offers page-cited chat, a notation glossary, a citation graph, a draft review and a multi-paper comparison. The design follows one rule: *the LLM extracts and Python decides*. Every score, delta and verdict is computed deterministically, and the whole system runs on the free tier of a commercial LLM API. On a gold set of 50 papers from nine ML areas (2,796 items), extraction reaches a core-field F1 of 0.60 with precision between 0.81 and 0.94 per core field; 90% of extracted items are verified against the paper and 1% are hallucinated. Reading each of the 246 extracted items that match no gold item shows that only 10 are wrong: most are correct items that a strict matcher misses, so the scores are lower bounds. Recall is the bottleneck, above all for hyperparameters in appendix tables (30% covered). In an earlier run, requiring a located quote lowered the hallucination rate from 10% to 2%, and a support check lowered it to 0%. We release the system, the evaluation package, the audit and an annotation guide.**

***Index Terms*—scientific information extraction, grounding, attribution, reproducibility, large language models, document parsing, claim verification**

I. Introduction

Reading a research paper well takes one to three hours, and much of that time is spent finding the same facts: the datasets, baselines, metrics, hyperparameters, compute and limitations. Judging whether the work can be reproduced is harder still. Surveys of the AI literature found that no sampled paper documented every factor needed for reproduction [1]. An attempt to re-implement 255 papers without their code succeeded only about two times in three [2]. Released code does not settle the question either, because it often uses settings other than those reported [3], [4].

LLMs are an obvious aid, but their output is fluent and unattributed, and they hallucinate facts [5]. Attribution research shows that generated statements must be checked against an identified source to be trusted [6], [7]. For scientific use, where a wrong learning rate silently ruins a replication, unverifiable output is of little value.

We present *PaperLens*, built around a single principle: **the LLM extracts, and Python decides**. The LLM's only job is to read text and return schema-validated items, each with a verbatim quote. Deterministic code does everything else. It locates every quote in the PDF, scores reproducibility, matches claims to table cells and recomputes their deltas, compares the paper's hyperparameters with the code, and draws every verdict. The system runs on the free tier of the Gemini API [8], so every design choice also minimises the number of LLM calls.

Our contributions are:

1) An end-to-end system that parses a paper with layout coordinates and extracts a fixed schema. Every item carries evidence that is located in the PDF and checked for support, and items that cannot be verified are labelled rather than dropped.
2) Three deterministic checks built on the extraction: a reproducibility scorecard, a claim verifier that recomputes claimed differences from parsed tables, and a static paper-vs-code comparison that never executes repository code.
3) An evaluation package with a documented matching scheme, a gold set of 50 papers from nine ML areas with 2,796 items, a hand audit of every extracted item that matches no gold item, and three ablations that need no LLM calls: evidence policy, parser and retrieval.
4) An honest assessment. Precision is high on every core field and errors are rare once matcher misses are separated out; the main weakness is recall on hyperparameters and tasks; and our reference annotations were produced with LLM assistance.

II. Related Work

A. Scientific information extraction

Entity and relation extraction from papers has been studied with dedicated datasets: SciERC for entities, relations and coreference [9], SciREX for document-level task/dataset/metric/method relations [10], and the leaderboard-construction work of Hou et al. [11]. AxCell extracts results

from machine-learning papers, using table segmentation [12]. These systems rely on fine-tuned encoders such as SciBERT [13]. Dagdelen et al. fine-tune LLMs for joint named-entity and relation extraction that outputs structured JSON for materials science [14]. PaperLens differs in two ways. It uses a general-purpose LLM without fine-tuning, and it makes evidence a first-class output that is checked against the source rather than an optional by-product.

B. Parsing scientific PDFs

GROBID turns PDFs into structured TEI with sections, references and coordinates [15], and it underlies corpora such as S2ORC [16]. Nougat converts page images to markup with a vision transformer [17], and Docling combines layout analysis with table structure recognition [18]. Table structure recognition has its own benchmarks [19], [20]. We found GROBID's table grids to be unreliable on our papers, so PaperLens rebuilds each grid from the PDF's word positions and uses GROBID for structure, sentences and citations (Section IV-A).

C. Grounded generation and attribution

Retrieval-augmented generation conditions an LLM on retrieved passages [21]. ALCE evaluates generated text whose statements cite their sources [7], and the AIS framework defines when a statement is attributable to an identified source [6]. FActScore breaks text into atomic facts and checks each against a knowledge source [22], while SelfCheckGPT detects hallucinations without external resources [23]. Long contexts are used unevenly by LLMs [24], which motivates our per-section extraction. PaperLens applies the attribution idea at the level of single extracted items and enforces it in code. A quote is accepted only if a fuzzy string match [25] finds it in the parsed paper.

D. Question answering and reading assistants

Qasper anchors information-seeking questions in research papers [26]. PaperQA [27] and OpenScholar [28] answer scientific questions with citations over large literature collections. Reading interfaces augment the paper itself. ScholarPhi shows position-sensitive definitions of terms and symbols [29], building on document-level definition detection [30], and the Semantic Reader project generalises these ideas [31]. TLDR generates extreme summaries of papers [32]. PaperLens works on a single paper. Its chat cites pages of that paper, and its notation glossary grounds each symbol's meaning in a sentence, in the spirit of ScholarPhi.

E. Scientific claim verification

SciFact verifies claims against abstracts [33]. TabFact [34] and SciTab [35] verify claims against tables, and SciTab shows that compositional reasoning over scientific tables remains hard for LLMs. We avoid asking the LLM for verdicts. It copies a claim's numbers, and Python finds the cells and recomputes the claimed difference.

F. Reproducibility

Beyond the surveys above [1], [2], the NeurIPS 2019 reproducibility programme introduced a checklist, a code policy and a reproducibility challenge [3]. Dodge et al. argue for reporting the expected validation performance as a function of the tuning budget [36], Bouthillier et al. separate methods reproducibility from inferential reproducibility [37], and Kapoor and Narayanan document data leakage across ML-based science [38]. Recent benchmarks ask agents to set up and run research code (SUPER [39], CORE-Bench [40]) or to replicate papers from scratch (PaperBench [41]). PaperLens takes the opposite and cheaper route. It never executes code, and it reports what a reproduction would need and where the paper and its code disagree.

G. LLM-assisted reviewing

Yuan et al. found generated reviews broad in coverage but less constructive and less factual than human ones [42]. Liang et al. report that GPT-4 feedback overlaps with human reviewers about as much as two humans overlap [43], and Zhou et al. find LLM reviewers unreliable for scoring [44]. PaperLens therefore offers a draft review without a score, and it lists facts established in Python separately from the LLM's opinions.

H. How PaperLens compares

Table I compares PaperLens with representative systems along the design properties that matter for trusting a paper analysis. Each prior line of work covers part of the problem well: SciREX and AxCell extract task, dataset and result tuples but need trained models and give no evidence per item; PaperQA and OpenScholar cite their sources but answer questions rather than fill a schema; SciTab and SciFact study verification but leave the verdict to a model; CORE-Bench and PaperBench check reproducibility by running code, at hours of agent time per paper. PaperLens is, to our knowledge, the only system in the table that (i) fills a full extraction schema including hyperparameters, (ii) attaches a quote to every item and checks it against the paper in code, (iii) leaves every numeric verdict to Python, and (iv) adds reproducibility, claim and paper-vs-code checks, all without fine-tuning and on a free-tier API. The table compares design, not accuracy: the systems solve different tasks on different data, so their reported scores are not comparable. The closest like-for-like comparison we can make is internal: the "keep all" arm of ablation A3 is what an LLM extractor without our evidence checks would deliver, and in our first run it hallucinated 10% of its items against 0% with the checks (Section VI).

III. System Overview

Fig. 1 shows the architecture. A Next.js interface uploads a paper or a link (arXiv, DOI, OpenReview, ACL Anthology, a direct PDF URL). A FastAPI service stores it and queues a job, and a Celery worker runs the pipeline, one paper at a time, writing to PostgreSQL. Progress is streamed to the browser with server-sent events. Every LLM call goes through a single

TABLE I

COMPARISON WITH REPRESENTATIVE FRAMEWORKS (✓ = CORE FEATURE, (✓) = PARTIAL OR INDIRECT, – = ABSENT OR NOT A GOAL). THE LAST COLUMN SAYS WHAT EACH NEEDS TO RUN. DESIGN COMPARISON ONLY: THE SYSTEMS ADDRESS DIFFERENT TASKS, SO ACCURACY NUMBERS ARE NOT COMPARABLE.

Framework	Schema extraction (incl. HPs)	Quote per item	Quote checked in code	Highlight in PDF	Numeric verdicts in code	Repro. check	Claims vs tables	Paper vs code	Halluc. measured	No fine-tuning	Needs
SciREX [10]	(✓)	(✓)	–	–	–	–	–	–	–	–	trained model
AxCell [12]	(✓)	–	–	–	–	–	(✓)	–	–	–	trained model
LLM-NERRE [14]	✓	–	–	–	–	–	–	–	–	–	fine-tuned LLM
PaperQA [27]	–	✓	(✓)	–	–	–	–	–	(✓)	✓	LLM API
OpenScholar [28]	–	✓	(✓)	–	–	–	–	–	(✓)	(✓)	LLM + paper datastore
ScholarPhi / Semantic Reader [29], [31]	–	✓	(✓)	✓	–	–	–	–	–	–	trained detectors
SciFact, SciTab [33], [35]	–	(✓)	–	–	–	–	✓	–	–	(✓)	model verdict
SUPER, CORE-Bench, PaperBench [39]–[41]	–	–	–	–	(✓)	✓	–	(✓)	–	✓	agent runs code (hours)
LLM reviewers [43]	–	–	–	–	–	(✓)	–	–	–	✓	LLM API
General LLM chat over a PDF	(✓)	(✓)	–	–	–	(✓)	(✓)	–	–	✓	LLM API
PaperLens (ours)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	free-tier API, CPU

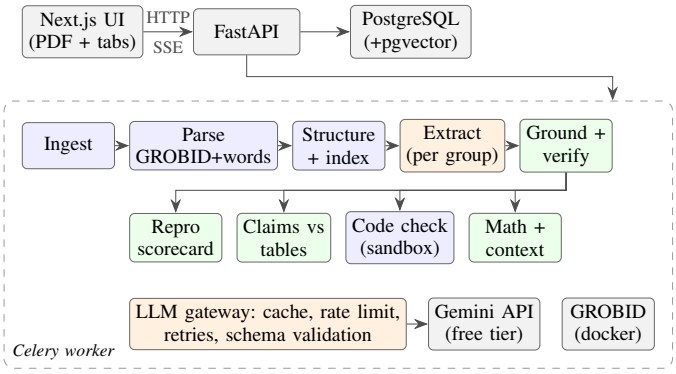


Fig. 1. Architecture. Orange stages call the LLM, blue stages are pure Python, and green stages make at most one LLM call whose output Python checks. The gateway is the only path to the LLM.

gateway. The gateway caches responses by (input, prompt version, model), rate-limits per model, retries with exponential backoff, switches to a backup model when one is overloaded, validates the output against a Pydantic schema with one repair attempt, and logs the provider, model and prompt version. Model names live only in configuration. Prompts are versioned templates, and results are keyed by (paper, stage, prompt version, model), so every stage is idempotent and a re-run of a cached paper costs nothing.

Table II lists the pipeline. Four of its stages make no LLM call at all, and extraction with verification uses about six calls per paper.

TABLE II

PIPELINE STAGES AND THEIR USE OF THE LLM.

Stage	LLM	What it does
Parse	none	GROBID TEI + PyMuPDF words → blocks with page boxes; table grids from word positions
Structure	none	section roles, paragraph chunks, BM25 + vector index
Extract	1/group	schema items, each with value, verbatim quote, chunk id
Verify	batched	quote located (fuzzy $\geq$ 90); support judged; unfound kept as *unlocated*
Repro	1	21-item checklist: rules first, LLM for the rest; Python score
Claims	1	LLM copies numbers; Python matches cells, recomputes, decides
Results	none	typed results tables, best value per column
Code	none	repository ranking, hardened clone, static scan, comparison
Math	1	LaTeX equations, grounded symbol glossary
Context	1	key references, citation graph, grounded novelty relations
Chat	1/turn	hybrid retrieval, cited answer or refusal
Review	1	draft review; findings supplied by Python; no score

IV. METHOD

A. Parsing and table reconstruction

GROBID provides sections, sentences with coordinates, references and resolved in-text citations. All coordinates are kept in PDF points, so any span can be highlighted later. Its table grids, however, were often wrong or placed on the wrong page. PaperLens therefore locates each table by its own caption ("Table 3:") and rebuilds the grid from the PDF's word positions. Columns come from whitespace gaps, wrapped cells are merged, bold cells are detected, and values such as 24.3 ± 0.2 are parsed. A grid that fails a sanity check is dropped, because no grid is better than a wrong grid. If GROBID is unavailable, a PyMuPDF-only parser takes over.

B. Grounded extraction

Sections are grouped (front matter, method, experiments, closing), and each group is sent in one call with a JSON schema. Paragraphs are labelled with their chunk ids. For every item the LLM returns a value, a verbatim quote and the chunk id the quote comes from. Python then locates each quote q. It first tries the cited chunk c and then the whole paper D, using the partial-ratio similarity $s(\cdot,\cdot)$ of the normalised strings, which is a Levenshtein-based measure [25]:

$$\hat{b}(q) = \arg\max_{b \in \{c\} \cup D} s(q,b), \qquad \text{located} \iff s\big(q, \hat{b}(q)\big) \geq \tau, \tag{1}$$

with $\tau = 90$. A located quote is mapped to sentence boxes for highlighting. A table quote is a row written as the prompt showed it ("cell | cell"), and it highlights that row. A single batched call then labels each located quote as *verified* (it supports the item), *partial* or *unsupported*. A quote that is not found is labelled *unlocated*. Such items are kept and badged, never silently dropped, so the user sees them and the evaluation counts them.

C. Reproducibility scorecard

A 21-item checklist, derived from the NeurIPS checklist [3], is organised into five groups g with weights W_g. The groups are Code (0.25), Data (0.20), Experimental setup (0.25), Compute (0.10) and Statistical reporting (0.20). Rules decide an item first, using regular expressions, the document structure and GitHub metadata. One LLM call judges the remaining items, and each judgement must quote the paper. An LLM "present" whose quote is not confirmed is capped at "partial". With points $p(\text{present}) = 1$, $p(\text{partial}) = 0.5$ and $p(\text{missing}) = 0$, and item weights w_i over the items S_g of group g that are scored (not unknown or not applicable), the score is

$$\text{score} = 100 \cdot \frac{\sum_g W_g \, \sigma_g}{\sum_g W_g}, \qquad \sigma_g = \frac{\sum_{i \in S_g} w_i \, p(s_i)}{\sum_{i \in S_g} w_i}, \quad (2)$$

where the outer sums run over groups with at least one scored item.

D. Claim verification

For each quantitative claim, the LLM copies the method, baseline, metric, dataset, the claimed value or difference δ^*, and its unit and comparator, exactly as written. Python finds the candidate cells x_m (method) and x_b (baseline) in the parsed results tables, recomputes $\hat{\delta} = x_m - x_b$ (or a ratio for factor claims) and decides. The claim is *supported* if $|\hat{\delta} - \delta^*| \leq \epsilon$, with $\epsilon = 0.15$ absolute, set in configuration. *Contradicted* requires exactly one unambiguous pair of cells. Ambiguous or missing matches give *unsupported*, so a table the parser got wrong cannot make a paper look wrong.

E. Paper vs code

Repository links are ranked from the paper text, telling the authors' own code apart from libraries the work is "based on". The repository is cloned shallowly, with hooks, symlinks and local protocols disabled and a size cap. A parse-only scanner then runs inside a sandbox container and reads Python ASTs, YAML, JSON, TOML and README flags. It never executes repository code. Assignments under inference-only branches are skipped. Hyperparameter names are aligned with a synonym table (`lr` ↔ `learning_rate`), and values are compared after numeric normalisation ($3\text{e}{-4} = 0.0003$).

F. Chat, notation and context

Chat retrieves paragraph passages with BM25 [45] and local dense embeddings [46], [47], fused by reciprocal rank fusion [48]:

$$\text{RRF}(d) = \sum_{r \in \{\text{bm25, dense}\}} \frac{1}{k + \text{rank}_r(d)}, \qquad k = 60. \quad (3)$$

Passages are labelled [P1], [P2], … in the prompt. Python removes citation markers the model invents, refuses an answer with no citation, and refuses without an LLM call when nothing relevant is retrieved. For notation, the arXiv LaTeX source is read in memory (never compiled). Equations are numbered by LaTeX's own rules so that they match the PDF, and one call builds a glossary whose symbol meanings are each grounded in a sentence. For context, key references are the ones cited most often in the text, with the citing sentences as evidence. Semantic Scholar adds citation counts and influential citations [49] when it is reachable.

G. Review, report and comparison

Reviewer mode asks the LLM for a summary, strengths, weaknesses, missing ablations and questions for the authors, each point quoted. Python adds findings from the scorecard, the claims and the code check, and gives no score. A Word report and a multi-paper comparison matrix are built from stored results without an LLM call.

V. Experimental Setup

A. Gold set

The evaluation set contains 50 arXiv papers from 2014 to 2024 (Table III). Ten *reference* papers were used during development: Bahdanau et al. [50], VGG [51], ResNet [52], Transformer [53], BERT [54], RoBERTa [55], DistilBERT [56], SimCLR [57], ViT [58] and LoRA [59]. Forty *new* papers were added afterwards and never used for development. They were chosen to balance nine areas and to vary the layout: appendix hyperparameter tables (LLaMA, DINOv2, Whisper), prompting papers with few settings (chain-of-thought, ReAct), theory-heavy papers (scaling laws, KAN) and papers without released code (Lottery Ticket, Chinchilla). The full list with arXiv identifiers is part of the release. The gold set holds 2,796 items in seven fields: task, datasets, baselines, metrics, hyperparameters, compute and code URLs. An annotation guide defines what counts in each field.

Annotation provenance. Because of time constraints, the reference annotations were produced by an LLM (Claude) reading each paper's full text page by page under the guide. Each part was adjudicated once after its first comparison with PaperLens. Items that the papers state and that fit the definitions were added (14 for the reference ten, 34 for the new forty), one over-broad alias was removed, no alias was written in the system's wording, and nothing was removed to suit the system. For the new forty, every extracted item that matched no gold item was read against the paper and labelled (Section VII). Every change is logged. Our numbers therefore

TABLE III
GOLD SET COMPOSITION (* = IN THE REFERENCE TEN).

Area	Papers	Papers (short names)
NLP	16	Bahdanau*, Transformer*, BERT*, RoBERTa*, DistilBERT*, LoRA*, T5, ELECTRA, SimCSE, RAG, chain-of-thought, ReAct, LLaMA, DPO, QLoRA, DeepSeekMath
CV	13	VGG*, ResNet*, SimCLR*, ViT*, Faster R-CNN, U-Net, Swin, MAE, ConvNeXt, DINOv2, SAM, ViT registers, SigLIP
ML	9	BatchNorm, AdamW, PGD, Lottery Ticket, scaling laws, Chinchilla, FlashAttention, Mamba, KAN
Generative	3	latent diffusion, DiT, flow matching
Graph / RL	2 / 2	GCN, GAT / PPO, SAC
Speech	2	wav2vec 2.0, Whisper
Time series	2	PatchTST, iTransformer
Multimodal	1	LLaVA

measure agreement with an *LLM-annotated* reference, and we discuss this threat in Section VIII-A.

B. Matching and metrics

Values are normalised (Unicode NFKD, number words turned into digits, decimals kept). A predicted text item matches a gold item when the two are nearly equal (fuzzy ≥ 88 with equal digit sequences, so CIFAR-10 never matches CIFAR-100), or when one contains the other and the remainder is filler, or when the contained gold phrase is specific on its own. An item that packs several values is tried part by part. Hyperparameters must agree on the name (through a synonym table) and on the parsed value ($6\% = 0.06$). For a field with predicted items P and gold items G,

$$\mathrm{P} = \frac{|\{p \in P : \exists g,\, p \sim g\}|}{|P|}, \quad \mathrm{R} = \frac{|\{g \in G : \exists p,\, p \sim g\}|}{|G|}, \tag{4}$$

and F1 is their harmonic mean (micro over papers; macro = mean of per-paper F1). The *hallucination rate* is the share of extracted items that are unlocated, unsupported or without evidence. *Numeric accuracy* is the share of extracted hyperparameters, among those whose name is in the gold, that have a correct value. *Hyperparameter coverage* is the share of gold (paper, name) pairs for which a correct value was extracted. Checklist agreement uses accuracy and Cohen's κ [60].

C. Configuration

Extraction used Gemini Flash-Lite through the free tier (October 2026) with the section-group prompts, GROBID 0.8.1 and the default thresholds above. The main numbers come from one run over all 50 papers (`collect_flash_lite`), made with the pipeline as of the last phase on a team member's laptop by a resumable collector that downloads each paper, checks its title and runs the pipeline one paper at a time. An earlier run over the reference ten (`phase2_flash_lite`, made when extraction was first built) is reported alongside. All tables and figures in this paper are regenerated by one command from the stored runs.

TABLE IV
EXTRACTION RESULTS PER FIELD (50 PAPERS, RUN `COLLECT_FLASH_LITE`).

Field	Gold	Pred	P	R	F1	$\mathrm{F1}_{\mathrm{macro}}$
Task	172	78	0.81	0.37	0.51	0.53
Datasets	490	320	0.94	0.63	**0.75**	0.79
Baselines	472	306	0.84	0.54	0.66	0.62
Metrics	239	171	0.89	0.62	0.73	0.74
Hyperparameters	1310	407	0.83	0.27	0.41	0.43
Compute	71	70	0.54	0.51	0.52	0.59
Code URLs	42	68	0.44	0.69	0.54	0.73
Core-field F1 (micro average of the five core fields)					**0.60**	
Hyperparameter coverage / numeric accuracy					30% / 91%	
Verified evidence / hallucination rate					90% / 1%	

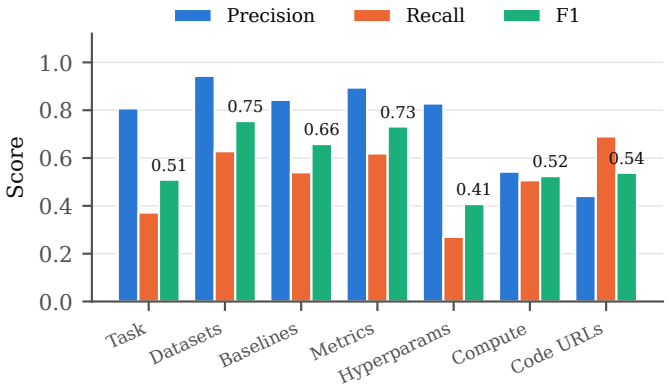


Fig. 2. Precision, recall and F1 per field (50 papers). Precision is high on the core fields, and recall limits F1.

VI. RESULTS

A. Extraction quality

Table IV and Fig. 2 report per-field scores over the 50 papers. Precision on the core fields lies between 0.81 and 0.94. When PaperLens reports a dataset, metric or baseline, it is usually right, and Section VII shows that most of the remaining "false positives" are correct items our matcher does not recognise. Recall is the bottleneck, above all for hyperparameters (0.27) and tasks (0.37). The micro-averaged core-field F1 is 0.60, short of our target of 0.80. Of the extracted hyperparameters whose name matched a gold name, 91% carried the right value, but only 30% of gold hyperparameters were covered. 90% of all extracted items are verified against the paper and 1% are unlocated or unsupported.

By paper set and area. Table V splits the run. The reference ten score a core-field F1 of 0.72 and the new forty 0.57. The gap is expected in part, since the prompts and thresholds were tuned while reading the reference papers, but it also reflects the new papers' layout: several state settings per model size in appendix tables (LLaMA, Whisper, T5). The graph papers are easiest (short, one table of settings); RL and generative-model papers are hardest, with hyperparameter coverage of 7%, because their settings sit in long appendix

TABLE V
CORE-FIELD F1 AND HYPERPARAMETER COVERAGE BY PAPER SET AND AREA (50 PAPERS).

Group	Papers	Core F1	HP coverage	Verified
Reference set	10	0.72	40%	88%
New set	40	0.57	28%	91%
NLP	16	0.61	32%	90%
CV	13	0.60	36%	88%
ML	9	0.51	32%	90%
Generative	3	0.56	7%	96%
Graph	2	0.82	59%	94%
RL	2	0.43	7%	82%
Speech	2	0.62	24%	95%
Time series	2	0.68	26%	93%
Multimodal	1	0.79	50%	94%

tables. With two or three papers per area, these per-area figures are indicative only. Fig. 3 shows every paper.

Run to run. On the reference ten, the 50-paper run matches the earlier run's core-field F1 (0.72 in both). Its hyperparameter coverage is close (40% vs 43%), and its numeric accuracy is 96% vs 100%. Its evidence is much cleaner: 88% of items verified instead of 80%, and a hallucination rate of 1% instead of 10%. The earlier run had 35 unlocated items against 2. The runs differ in machine, GROBID instance, sampling and pipeline version (later phases changed shared code), and we have not isolated which difference matters.

B. Evidence policy (ablation A3)

Because every item carries a status, we can score the same extraction under stricter evidence policies without new LLM calls (Fig. 4). In the earlier run, requiring the quote to be found in the paper lowers the hallucination rate from 10% to 2%, and additionally requiring the support check lowers it to 0%, while precision stays at 0.96 to 0.97. The cost is recall, which falls from 0.54 to 0.43 under the strictest policy, because some correct items come with a paraphrased quote that cannot be located. In the 50-paper run almost every quote is located, so the policies barely differ: the strictest one removes the last 1% of hallucinated items and lowers recall from 0.42 to 0.39. We keep all items in the interface and badge their status. The reader sees the evidence level, and the metrics still count every item.

C. Parser (ablation A1) and retrieval (ablation A6)

Fig. 5 compares GROBID with a PyMuPDF-only parse on all 50 papers by the share of gold values still present in the parsed text, which is an upper bound on extraction recall. Raw PyMuPDF text keeps more of the values (90% vs 83%), mostly numbers in tables, footnotes and appendix lists. GROBID, however, recovers the structure the rest of the pipeline depends on: 1,520 vs 1,165 section headings, 3,026 vs 2,394 references, 580 vs 243 equations, and in-text citations. The gap is largest for hyperparameters: a third of the gold values (33%) never reach the extractor through GROBID's text, against 19% with PyMuPDF. Since 67% is the ceiling for hyperparameter coverage and PaperLens reaches 30%, the extractor recovers less than half of what the parse makes available, and a parse that keeps table and appendix text would raise the ceiling itself.

TABLE VI
ABLATION A6: RETRIEVAL RECALL@5 FOR CHAT (50 PAPERS, GROBID PARSE).

Retrieval	All	Task	Datasets	Baselines	Metrics	HPs
BM25	**45%**	**53%**	36%	**41%**	48%	50%
Dense	36%	19%	37%	29%	46%	41%
Hybrid	**45%**	45%	**40%**	38%	**49%**	**51%**

TABLE VII
ACCEPTANCE CHECKS OF THE REMAINING COMPONENTS.

Component	Criterion	Result
Parsing	10 arXiv papers; tables on ≥ 8	10/10
Extraction	full output ≤ 3 min	10/10; 15–50 s per paper
Repro. scorecard	$\geq 80\%$ agreement	75%, $\kappa = 0.57$ (below target)
Code discovery	$\geq 80\%$ repos found	8/8
Paper vs code	one real mismatch	BERT (below)
Chat / math / context	cited answers; glossary on 5 papers; graph	passed (VAE, Adam, DDPM, Transformer, LoRA)

For chat (A6), we ask one question per field and per hyperparameter name and count a gold item as found when it occurs in the top five passages (2,086 question–item pairs, Table VI). BM25 and hybrid retrieval tie at 45% recall@5; dense retrieval alone is clearly worse (36%), above all on task questions (19% vs 53%), where the answer shares words with the question ("task", the paper's own terms) rather than meaning. Hybrid fusion helps on datasets, metrics and hyperparameters and costs on tasks, which supports keeping BM25 in the chat's fusion rather than relying on embeddings.

D. Reproducibility, code and other checks

Table VII summarises the acceptance checks of the other components. The scorecard agreed with the team's annotations on 158 of 210 checklist decisions (75%, $\kappa = 0.57$). This is below the targets of 80% and $\kappa = 0.6$, but up from 65% before item-level guidance was added. The largest disagreements concern training time, re-run baselines and README instructions. Repository discovery found the official code for all eight papers that have it.

Case study: BERT. The paper states that pre-training used a learning rate of 10^{-4}, a batch size of 256 and 1M steps [54]. The static scan of the official repository found the defaults of `run_pretraining.py` to be 5×10^{-5}, 32 and 100,000. PaperLens reports this as a mismatch and shows the paper's sentence beside the code location. The difference is legitimate, since the script's values are command-line defaults, but anyone running the script as is would not reproduce the paper. This is exactly the gap the check is meant to surface.

The phase-5 check covered five math-heavy papers (VAE [61], Adam [62], DDPM [63], Transformer, LoRA).

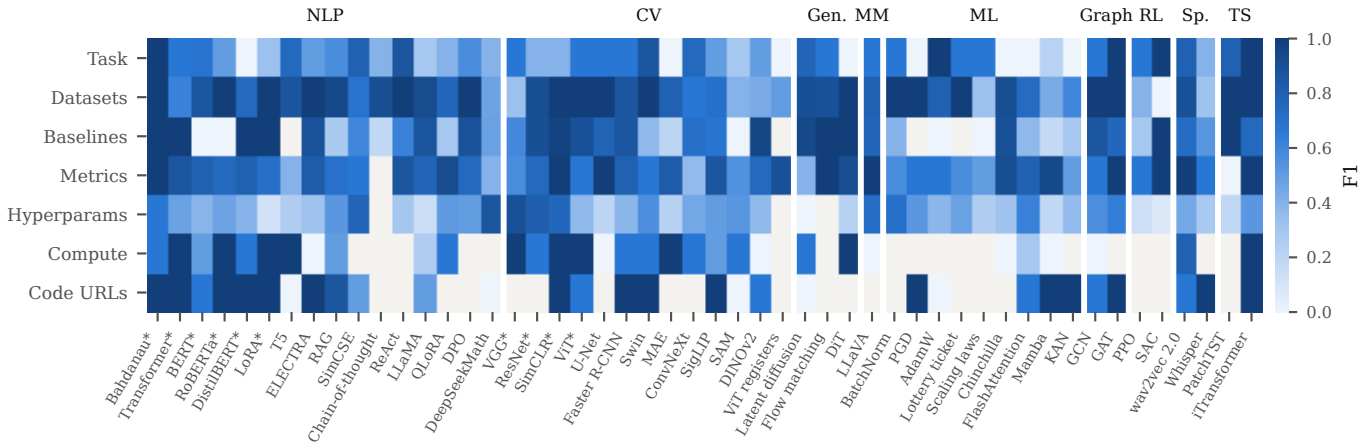


Fig. 3. Per-paper F1 by field, papers grouped by area (* = reference set; grey = no gold item in that field, e.g. no released code). Gen. = generative models, MM = multimodal, Sp. = speech, TS = time series. Hyperparameters and tasks are the weak rows.

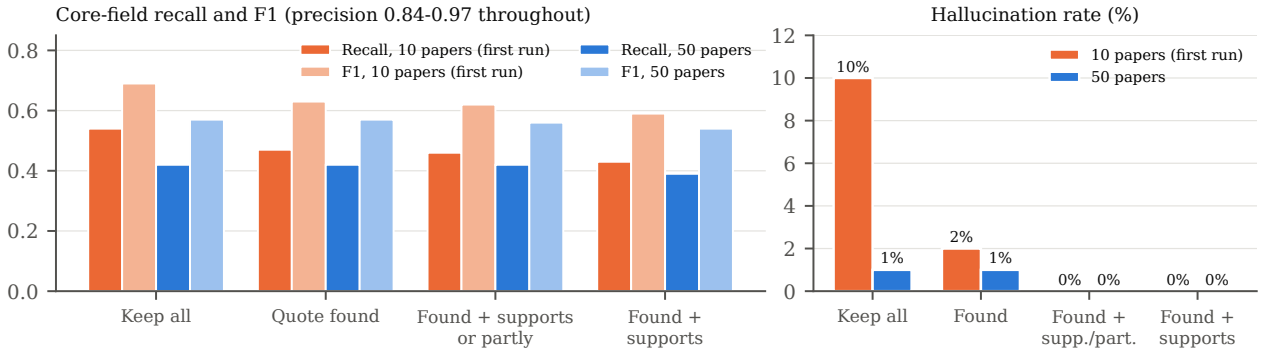


Fig. 4. Ablation A3: each run scored under four evidence policies (core fields). Left: recall and F1. Right: hallucination rate. Requiring a located, supporting quote removes hallucinated items at a moderate cost in recall.

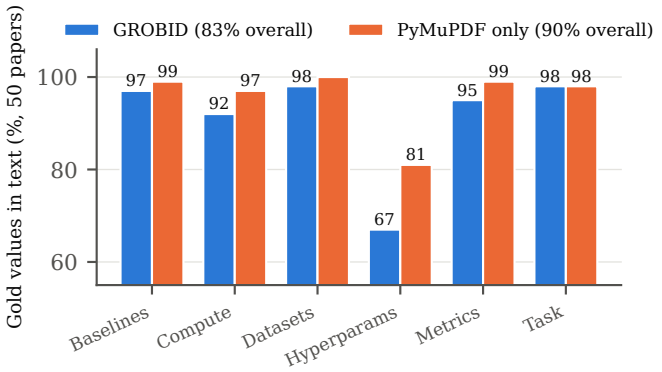


Fig. 5. Ablation A1: gold values still present in the parsed text, by field (50 papers). GROBID loses a third of the hyperparameter values, which caps extraction recall.

Chat answers cited pages, off-topic questions were refused, every glossary symbol's quote was verified, and the citation graph rendered.

E. Interface

Fig. 6 shows the workspace. Clicking an evidence chip scrolls the PDF to the quoted sentence and highlights it in the colour of its status. Fig. 8 shows the comparison matrix, built from the stored extractions of two papers without an LLM call. Each cell gives the page where the paper states the item.

VII. Error Analysis

The 50-paper run missed 1,602 of 2,796 gold items, 956 of them hyperparameters, and produced 239 items that match no gold item (Fig. 9).

What the unmatched items are. We read each of the 246 unmatched items of the new forty papers against the paper (the audit is released with the gold set). Only 10 are wrong: four model families listed as datasets (QLoRA's OPT, LLaMA, BLOOM, Pythia), a method listed as its own baseline, two metrics the paper does not report, two values the paper does not state, and one code link not given in the text. 127 are correct but missed by the matcher, 31 were gold omissions (added during adjudication), and 78 are out of scope.

- **Matcher misses (127)** come from two sources. One is hyperparameter names: "lambda" vs

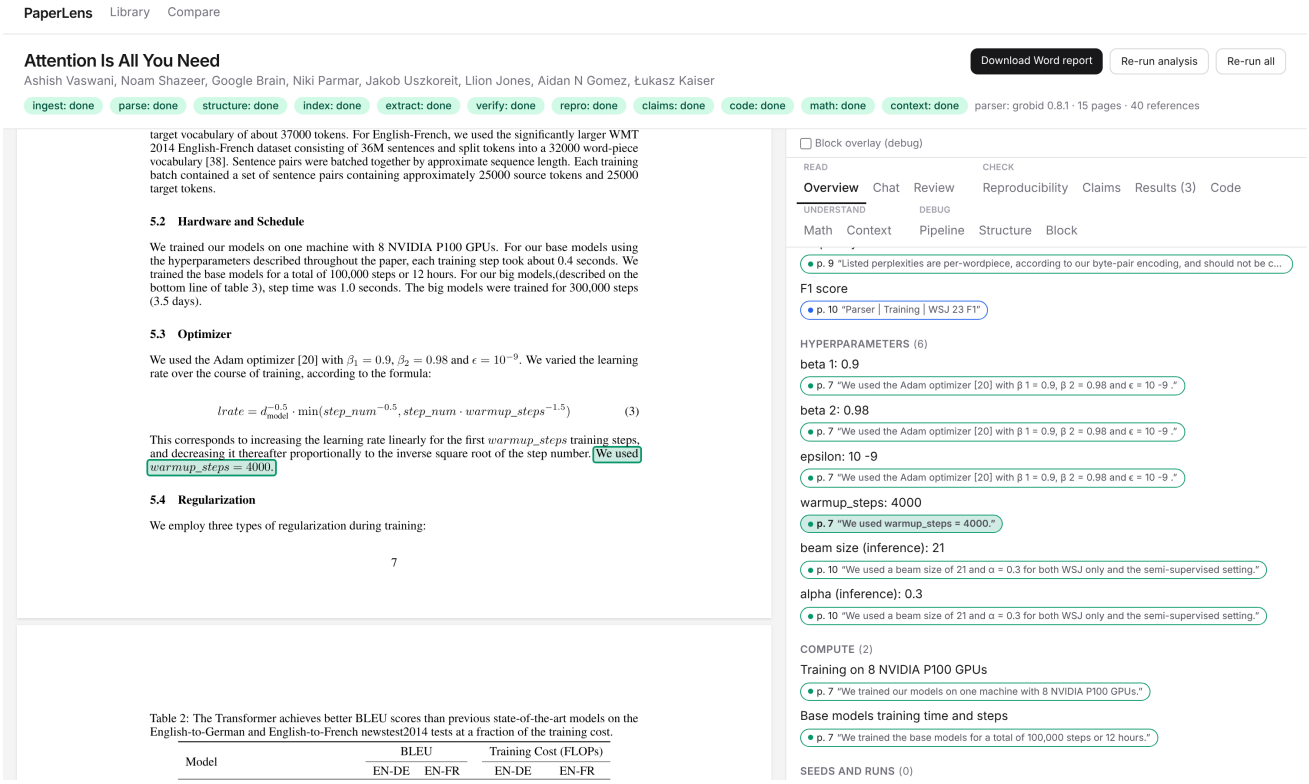


Fig. 6. The PaperLens workspace on the Transformer paper [53], with items from the evaluated Gemini run. Selecting the evidence for `warmup_steps: 4000` highlights the source sentence on p. 7 (left). Chip colours encode the evidence status (green = verified, blue = unchecked).

Table 2 The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fractio p. 8 CSV

Method	BLEU EN-DE ↑	BLEU EN-FR ↑	Training Cost (FLOPs) EN-DE ↓	Training Cost (FLOPs) EN-FR ↓
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		1.0 · 10^20
GNMT + RL [38]	24.6	39.92	2.3 · 10^19	1.4 · 10^20
ConvS2S [9]	25.16	40.46	9.6 · 10^18	1.5 · 10^20
MoE [32]	26.03	40.56	2.0 · 10^19	1.2 · 10^20
Deep-Att + PosUnk Ensemble [39]		40.4		8.0 · 10^20
GNMT + RL Ensemble [38]	26.30	41.16	1.8 · 10^20	1.1 · 10^21
ConvS2S Ensemble [9]	26.36	41.29	7.7 · 10^19	1.2 · 10^21
Transformer (base model)	27.3	38.1		3.3 · 10^18
Transformer (big)	28.4	41.8		2.3 · 10^19

Fig. 7. A results table rebuilt from word positions (Transformer, Table 2). The best value per column is computed in Python from the header's direction (higher BLEU and lower training cost are better).

	Attention Is All You Need	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
OVERVIEW		
Task	machine translation; English constituency parsing p. 1	question answering; language inference p. 1
Compute	Training on 8 NVIDIA P100 GPUs; Base models training time and steps p. 7	Training time on a single Cloud TPU p. 5
Code	https://github.com/tensorflow/tensor2tensor p. 10	https://github.com/google-research/bert; https://github.com/tensorflow/tensor2tensor p. 2, 3
DATASETS		
WMT 2014 English-to-German	✓ p. 8	–
GLUE	–	✓ p. 5
SQuAD v1.1	–	✓ p. 6
TriviaQA	–	✓ p. 7
SQuAD 2.0	–	✓ p. 7
SWAG	–	✓ p. 7
CoNLL-2003	–	✓ p. 9
METRICS		
F1 score	✓ p. 10	✓ p. 6
BLEU	✓ p. 8	–
Perplexity	✓ p. 9	–
Accuracy	–	✓ p. 6
Exact Match (EM) and F1	–	✓ p. 7
Spearman correlations, F1, and accuracy	–	✓ p. 6
TRAINING		
learning rate	–	5e-5, 4e-5, 3e-5, and 2e-5 (fine-tuning); 5e-5 (fine-tuning); 2e-5 (fine-tuning) p. 6, 7
batch size	–	32 (fine-tuning); 48 (fine-tuning); 16 (fine-tuning) p. 6, 7
epochs	–	3 (fine-tuning); 2 (fine-tuning) p. 6, 7
warmup steps	4000 p. 7	–

Fig. 8. Comparison matrix for two papers (excerpt): tasks, compute, code, datasets and metrics with the page each is stated on. Rows shared by both papers are tinted.

`loss_weight_lambda` with alias "λ", "number of hidden units" vs "hidden units", or a Greek letter the normaliser drops. The other is values the extraction wrote in a broken form, such as "3.0e^−^4", "8e -4" or "2 24" for 2^{24}, which come from superscripts in the PDF text layer. A dedicated value normaliser and a richer synonym table would recover most of them.

- **Out of scope (78)**: third-party links (libraries, dataset pages, other papers' code; 35 of the 36 unmatched code URLs) and compute descriptions without hardware or time ("hardware used for training models").

Counting matcher misses and gold omissions as correct, 10 of the 246 items (4%) are wrong, so precision on the new forty is far higher than Table IV reports. This reading was done by the same LLM annotator and is part of the threat discussed in Section VIII-A.

What is missed. Five causes recur:

1) **Appendix and table hyperparameters (dominant).** Large-model papers state settings per model size in appendix tables: LLaMA's dimension, heads, layers and learning rate for four sizes; Whisper's five sizes; T5's five variants. The extraction usually returns one size. Long appendix groups dilute attention [24], and asking

for a quote per item seems to favour prose over table rows.

2) **Tasks listed too narrowly.** The model names one or two tasks, whereas the gold lists every task the experiments address (for example, SAM's edge detection, object proposals and text-to-mask experiments).
3) **Long benchmark lists.** Papers evaluated on large suites lose most of the list: DINOv2 misses 29 of its 42 gold datasets, SAM 20 of 27 and Whisper 17 of 21, while LLaMA, whose benchmarks are named in running text, misses only 4 of 27.
4) **Baselines cited by author name only,** such as T5's "previous best" results given as citations. Our scorer also undercounts one case: "Krizhevsky et al." does not match "Krizhevsky et al. 2012" because digit sequences must be equal.
5) **Parsing ceiling.** 17% of gold values, and 33% of hyperparameter values, do not survive GROBID's text at all (A1).

VIII. Discussion and Limitations

Precision by construction, recall by effort. Making the LLM quote its sources, and checking every quote in code, yields high precision and a measurable hallucination rate that can be driven to zero. Recall, however, depends on the model reading every table and appendix. Table-aware extraction of appendix settings is the most promising next step.

Deterministic decisions. Scores, deltas, verdicts and code comparisons are computed in Python. They are reproducible, testable with unit tests, and conservative by design: an ambiguous table match yields "unsupported", never "contradicted".

Limitations.

- The gold set has 50 papers but a single annotator (an LLM), with no double annotation and no inter-annotator agreement. Areas other than NLP, CV and ML have only one to three papers each.
- The scorecard's agreement with the team is below target.
- Cloning repositories failed on the team's Windows machines, so the code comparison was demonstrated on Linux. Repository discovery itself needs no clone.
- Semantic Scholar often rate-limits, and the context view then falls back to a PDF-only citation graph.
- Several planned ablations are pending:
  - extraction granularity (A2);
  - rule hints (A4);
  - a stronger model (A5);
  - LLM vs Python claim verdicts (A7);
  - extraction without quotes (full A3);
  - LaTeX and Docling parsers (A1);
  - Qasper [26].

A. Threats to validity

The reference annotations, and the audit of unmatched items, were produced with an LLM and adjudicated once against this system's output. Where the annotating LLM and Gemini share blind spots, recall may be overestimated. Where the matcher is strict, scores are underestimated, and the audit shows by how much. Adjudication only added items and never removed any to favour the system, and every change is logged. A human spot-check of two or three papers is in progress (TEAM: report the outcome). Each result comes from a single run of one model, the papers are well-known ML papers whose text the LLM may have seen in training, and absolute numbers may not transfer to recent or out-of-domain papers.

IX. Ethics and Privacy

Paper text is sent to a free-tier API whose inputs may be used to improve the provider's models. The interface warns users not to upload unpublished or confidential papers. Repository code is never executed. Reviewer mode gives no score, labels its output as an LLM draft, and separates it from facts computed in Python, consistent with evidence that LLM reviews should support human reviewers rather than replace them [43], [44].

X. Conclusion

PaperLens shows that a free-tier LLM can support careful paper reading when it is asked only to extract and quote, while deterministic code locates every quote and makes every decision. On 50 papers from nine areas it extracts with high precision (only 4% of the items the gold set lacks are wrong when read by hand) and 91% numeric accuracy on matched hyperparameters. Every item is traceable to a highlighted sentence, 90% are verified, and requiring evidence eliminates hallucinated items. Future work will target recall on appendix tables and per-size settings, normalise values written with superscripts, complete the pending ablations, and add independent human annotation.

Acknowledgment

The authors thank their project guide for feedback. PaperLens uses GROBID, PyMuPDF and the Gemini API free tier.

References

[1] O. E. Gundersen and S. Kjensmo, "State of the art: Reproducibility in artificial intelligence," in *Proc. AAAI Conf. Artificial Intelligence*, 2018, pp. 1644–1651.
[2] E. Raff, "A step toward quantifying independently reproducible machine learning research," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
[3] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.
[4] C. Collberg and T. A. Proebsting, "Repeatability in computer systems research," *Communications of the ACM*, vol. 59, no. 3, 2016.
[5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.
[6] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter, "Measuring attribution in natural language generation models," *Computational Linguistics*, vol. 49, no. 4, pp. 777–840, 2023.
[7] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 6465–6488.
[8] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

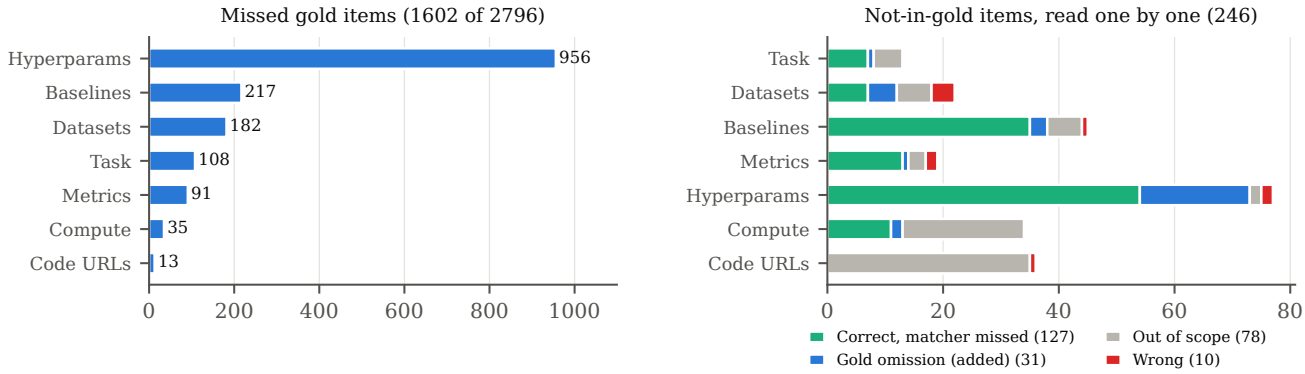


Fig. 9. Left: missed gold items by field (50 papers). Right: the 246 extracted items of the new forty papers that match no gold item, labelled by reading each against the paper.

[9] Y. Luan, L. He, M. Ostendorf, and H. Hajishirzi, "Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[10] S. Jain, M. van Zuylen, H. Hajishirzi, and I. Beltagy, "SciREX: A challenge dataset for document-level information extraction," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2020.

[11] Y. Hou, C. Jochim, M. Gleize, F. Bonin, and D. Ganguly, "Identification of tasks, datasets, evaluation metrics, and numeric scores for scientific leaderboards construction," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2019, pp. 5203–5213.

[12] M. Kardas, P. Czapla, P. Stenetorp, S. Ruder, S. Riedel, R. Taylor, and R. Stojnic, "AxCell: Automatic extraction of results from machine learning papers," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 8580–8594.

[13] I. Beltagy, K. Lo, and A. Cohan, "SciBERT: A pretrained language model for scientific text," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[14] J. Dagdelen, A. Dunn, S. Lee, N. Walker, A. S. Rosen, G. Ceder, K. A. Persson, and A. Jain, "Structured information extraction from scientific text with large language models," *Nature Communications*, vol. 15, p. 1418, 2024.

[15] P. Lopez, "GROBID: Combining automatic bibliographic data recognition and term extraction for scholarship publications," in *Research and Advanced Technology for Digital Libraries (ECDL)*, ser. Lecture Notes in Computer Science, vol. 5714, 2009.

[16] K. Lo, L. L. Wang, M. Neumann, R. Kinney, and D. S. Weld, "S2ORC: The semantic scholar open research corpus," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2020.

[17] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[18] C. Auer *et al.*, "Docling technical report," arXiv:2408.09869, 2024.

[19] X. Zhong, E. ShafieiBavani, and A. J. Yepes, "Image-based table recognition: Data, model, and evaluation," in *Proc. European Conf. Computer Vision (ECCV)*, 2020.

[20] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards comprehensive table extraction from unstructured documents," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 4634–4642.

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[22] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[23] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[24] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[25] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[26] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner, "A dataset of information-seeking questions and answers anchored in research papers," in *Proc. Conf. North American Chapter Assoc. Computational Linguistics (NAACL-HLT)*, 2021.

[27] J. Lála *et al.*, "PaperQA: Retrieval-augmented generative agent for scientific research," arXiv:2312.07559, 2023.

[28] A. Asai *et al.*, "Synthesizing scientific literature with retrieval-augmented language models," *Nature*, 2026, doi:10.1038/s41586-025-10072-4.

[29] A. Head, K. Lo, D. Kang, R. Fok, S. Skjonsberg, D. S. Weld, and M. A. Hearst, "Augmenting scientific papers with just-in-time, position-sensitive definitions of terms and symbols," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2021.

[30] D. Kang, A. Head, R. Sidhu, K. Lo, D. S. Weld, and M. A. Hearst, "Document-level definition detection in scholarly documents: Existing models, error analyses, and future directions," in *Proc. First Workshop on Scholarly Document Processing (SDP)*, 2020, pp. 196–206.

[31] K. Lo, J. C. Chang, A. Head, J. Bragg, A. X. Zhang, C. Trier *et al.*, "The semantic reader project: Augmenting scholarly documents through AI-powered interactive reading interfaces," *Communications of the ACM*, vol. 67, no. 10, 2024.

[32] I. Cachola, K. Lo, A. Cohan, and D. S. Weld, "TLDR: Extreme summarization of scientific documents," in *Findings of the Assoc. Computational Linguistics: EMNLP*, 2020.

[33] D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan, and H. Hajishirzi, "Fact or fiction: Verifying scientific claims," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[34] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang, "TabFact: A large-scale dataset for table-based fact verification," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[35] X. Lu, L. Pan, Q. Liu, P. Nakov, and M.-Y. Kan, "SCITAB: A challenging benchmark for compositional reasoning and claim verification on scientific tables," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 7787–7813.

[36] J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith, "Show your work: Improved reporting of experimental results," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[37] X. Bouthillier, C. Laurent, and P. Vincent, "Unreproducible research is reproducible," in *Proc. Int. Conf. Machine Learning (ICML)*, 2019.

[38] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[39] B. Bogin, K. Yang, S. Gupta, K. Richardson, E. Bransom, P. Clark, A. Sabharwal, and T. Khot, "SUPER: Evaluating agents on setting up and executing tasks from research repositories," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 12 622–12 645.

[40] Z. S. Siegel, S. Kapoor, N. Nadgir, B. Stroebl, and A. Narayanan, "CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark," *Transactions on Machine Learning Research*, 2024.

[41] G. Starace *et al.*, "PaperBench: Evaluating AI's ability to replicate AI research," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025.

[42] W. Yuan, P. Liu, and G. Neubig, "Can we automate scientific reviewing?" *Journal of Artificial Intelligence Research*, vol. 75, pp. 171–212, 2022.

[43] W. Liang, Y. Zhang *et al.*, "Can large language models provide useful feedback on research papers? a large-scale empirical analysis," *NEJM AI*, vol. 1, no. 8, 2024.

[44] R. Zhou, L. Chen, and K. Yu, "Is LLM a reliable reviewer? a comprehensive evaluation of LLM on automatic paper reviewing tasks," in *Proc. Joint Int. Conf. Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024, pp. 9340–9351.

[45] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[46] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[47] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[48] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009.

[49] M. Valenzuela, V. Ha, and O. Etzioni, "Identifying meaningful citations," in *AAAI Workshop on Scholarly Big Data: AI Perspectives, Challenges, and Ideas*, 2015.

[50] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[51] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[52] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.

[53] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[54] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American Chapter Assoc. Computational Linguistics (NAACL-HLT)*, 2019.

[55] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[56] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[57] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020.

[58] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[59] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[60] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[61] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2014.

[62] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[63] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.